

Institute of Plasma Research
Bhat,Gandhinagar

FPGA based Programmable Sweep Generator



Submitted by:

Oshi Teotia
T. Kavya Srri

June 2025

Contents

1	Introduction	1
1.1	Objective	1
1.1.1	Key Features	2
1.2	Hardware and Software	2
2	Understanding the Problem Statement	4
2.1	Background	4
2.2	Scope of Work	5
3	Design and Implementation	6
3.1	Counter Implementation	6
3.1.1	Purpose of the Counter	6
3.1.2	Integration with Signal Generation Logic	7
3.1.3	Verilog Code for Counter	7
3.2	ISE 14.7 and Vivado	8
3.2.1	Xilinx Vivado	8
3.3	Testbench	9
3.3.1	Testbench for Counter	9
3.3.2	Testbench for Signal Generation Logic	9
3.3.3	Simulation and Debugging in ISE 14.7	10
3.3.4	Conclusion	10
3.4	Study of Language Templates and AD5684 Datasheet	11
3.4.1	Verilog Language Templates	11
3.4.2	AD5684 Datasheet	11
3.5	SPI Communication for DAC Control	13
4	Key Concepts and Implementations	14
4.1	LIFO and FIFO	14
4.1.1	Concepts of LIFO and FIFO	14
4.1.2	Depth Calculations	14
4.2	Clock Division	15
4.3	Timing Diagram of DAC	15
4.4	Waveform Logic and Ramp Generation	15
4.4.1	Verilog Code for the Programmable Sweep Generator	16
4.4.2	Observation of Ramp Signal	22

5	Python GUI Development	24
5.1	GUI Overview	24
5.2	Control Parameters	24
5.3	Integration of FPGA and GUI	25
5.3.1	Python Code Example	26
6	Results and Observations	30
6.1	Ramp Signal Observation	30
6.2	System Performance	30
7	Conclusion	33
7.1	Summary of Accomplishments	33
7.2	Challenges Faced and Solutions Implemented	33
7.3	Future Work and Improvements	34
7.4	Final Thoughts	34
7.5	Challenges Faced	35
7.6	Future Work	35
8	References	36

June 21, 2025

Chapter 1

Introduction

1.1 Objective

The primary objective of this project is to design and implement a programmable sweep generator based on **FPGA (Field-Programmable Gate Array)** utilizing the **Spartan-6 FPGA** board from Xilinx. The system aims to generate a **ramp signal** whose key parameters—**frequency**, **amplitude**, **duty cycle**, and **offset**—can be **dynamically controlled** by the user. This control is facilitated through a **Graphical User Interface (GUI)** developed using **Python**.

The sweep generator is intended for applications where precise, adjustable signal waveforms are required, such as in signal testing, circuit characterization, and other electronic measurements. By leveraging the flexibility of the FPGA, the generator can produce **accurate ramp signals** over a wide frequency range and adjust other characteristics such as the amplitude and offset in real-time. This makes the system ideal for use in environments that require high-speed signal processing and quick adjustments without the need for hardware modifications.

The FPGA-based design offers several advantages over traditional methods of signal generation. Notably, it provides a **high degree of customization**, allowing the user to modify parameters on-the-fly, and **superior timing accuracy** since the FPGA's parallel processing capabilities ensure that the signal generation is synchronized and precise.

Additionally, the system's flexibility extends beyond the FPGA hardware itself, with the integration of a user-friendly Python-based GUI that allows users to interact with the signal generator in an intuitive manner. This GUI enables the real-time control of the signal parameters, with immediate feedback from the FPGA system, making the device accessible to both novice users and professionals who require fine-grained control over the generated waveforms.

1.1.1 Key Features

- **Ramp Signal Generation:** The core functionality is to generate a ramp signal whose frequency, amplitude, duty cycle, and offset are controllable.
- **Real-Time Parameter Control:** Through the GUI, users can adjust the key signal parameters on-the-fly.
- **Integration with FPGA:** The signal generation is performed on the Spartan-6 FPGA, ensuring high-speed, reliable performance.
- **DAC Control via SPI:** The system uses an **AD5684 DAC** to convert the digital signal from the FPGA into an analog waveform, with the FPGA controlling the DAC via **SPI (Serial Peripheral Interface)** communication.
- **Python-Based GUI:** The GUI, developed in Python, provides a simple interface for users to set the frequency, amplitude, duty cycle, and offset values.

1.2 Hardware and Software

The successful implementation of the FPGA-based programmable sweep generator requires a combination of specialized hardware components and software tools. Below, the key hardware and software components used in the project are described in detail.

- **FPGA Board: Spartan-6**
 - The Spartan-6 FPGA from Xilinx is at the heart of the system. It offers a balance between performance, resource availability, and cost, making it ideal for implementing signal processing and control systems.
 - The Spartan-6 family is designed for low power consumption and high-speed logic, which makes it well-suited for generating ramp signals in real-time.
 - The FPGA handles the core signal generation and control, including clock division, SPI communication, and the logic required to drive the DAC (AD5684).
 - The FPGA operates at a clock frequency of 25 MHz, providing enough processing speed for real-time control of the ramp signal parameters.
- **DAC: AD5684, with SPI Communication**
 - The AD5684 is a 16-bit Digital-to-Analog Converter (DAC) that receives digital data from the FPGA and converts it into an analog output. The high resolution of the DAC (16-bit) ensures precise control over the amplitude of the generated signal.
 - It communicates with the FPGA via the **SPI (Serial Peripheral Interface)** protocol, which allows fast and reliable data transfer between the FPGA and DAC. The SPI interface provides the ability to update the DAC's input values dynamically, ensuring real-time control of the output signal.
 - The DAC operates at a frequency of 50 MHz, allowing for rapid updates to the signal output to meet the required sweep generation speed.

- **Software: ISE 14.7 (Xilinx), Python for GUI Development**

- **ISE 14.7 (Xilinx):** Xilinx ISE 14.7 is a comprehensive FPGA design suite used for synthesis, simulation, and implementation of FPGA designs. The suite was used to write, synthesize, and deploy the Verilog code to the Spartan-6 FPGA, managing the design flow from RTL (Register Transfer Level) code to bitstream generation.
- **Python for GUI Development:** Python was chosen for the development of the graphical user interface (GUI) because of its ease of use and availability of powerful libraries. Libraries like Tkinter (for creating the interface) and PySerial (for serial communication with the FPGA) were used to implement the GUI. The Python interface allows users to control the key signal parameters such as frequency, amplitude, duty cycle, and offset, offering a user-friendly experience for real-time interaction with the FPGA.

- **Clock Frequencies**

- The system operates with a **25 MHz clock** for the FPGA. This clock drives the main signal generation logic and provides the timing required for controlling the ramp signal's parameters.
- The DAC can operate up to a frequency of **50 MHz**.
- The generated ramp signal was of **1 KHz frequency** .

Chapter 2

Understanding the Problem Statement

2.1 Background

The need for a **programmable sweep generator** is widespread in various domains such as **electronic circuit testing**, **signal processing**, and **instrumentation**. Traditional signal generators often come with fixed capabilities or are limited in their ability to produce diverse waveforms. This project aims to bridge this gap by developing an FPGA-based **sweep generator** capable of producing a **ramp signal** with adjustable parameters, such as frequency, amplitude, duty cycle, and offset.

The main challenge of this project was to design a **flexible system** that allows users to **control the signal generation** parameters dynamically. The flexibility of FPGA technology provides a perfect solution, allowing for hardware-accelerated signal generation with precise timing and synchronization. However, the complexity arises in ensuring that the FPGA-generated signals are adjustable in real-time and that they can be easily controlled by an external user interface.

The design of the sweep generator needed to ensure:

- **Variable Parameters:** The frequency, amplitude, duty cycle, and offset of the ramp signal must be easily adjusted by the user.
- **Real-time Control:** The system must allow users to modify parameters and see immediate changes in the output waveform.
- **User-Friendly Interface:** While the FPGA provides the hardware foundation, a **graphical user interface (GUI)** was needed to make it easy for users to interact with the system and adjust parameters in real-time.

This project thus aims to create a system that combines the power of FPGA with an intuitive Python-based GUI, allowing users to generate and control ramp signals across a wide frequency range.

2.2 Scope of Work

The project is focused on designing and implementing a **programmable sweep generator** system using FPGA. The following key tasks were undertaken to ensure the successful realization of this project:

- **FPGA Design Using Verilog:** The core signal generation logic was designed and implemented in **Verilog**, a hardware description language. This involved creating a modular system that generates ramp signals based on input parameters such as frequency, amplitude, and offset. The Verilog code was optimized for use on the Spartan-6 FPGA to ensure high performance and low latency in signal generation.
- **Clock Division for Synchronization:** The clock division logic was used to generate a ramp signal of desired frequency. The frequency was controlled using GUI.
- **Serial Communication Using SPI:** The FPGA communicates with the **AD5684 DAC** through the **SPI (Serial Peripheral Interface)** protocol. The FPGA was programmed to send control signals to the DAC, updating its internal register values to set the frequency, amplitude, and other parameters of the ramp signal. This required the implementation of an efficient SPI driver in Verilog to facilitate fast data transfer between the FPGA and the DAC.
- **Development of a Python-Based GUI:** A Python-based **Graphical User Interface (GUI)** was developed using the **Tkinter** library. The GUI allows users to easily control the parameters of the sweep generator, such as **frequency**, **amplitude**, **duty cycle**, and **offset**, by simply interacting with sliders and input fields. The GUI communicates with the FPGA via the SPI interface to update the DAC settings in real-time. Python was chosen due to its simplicity, flexibility, and support for various hardware communication protocols.
- **Testing and Validation:** The system's performance was thoroughly tested by generating different ramp signals and observing their behavior on an oscilloscope. The waveform characteristics were verified against the desired specifications, and adjustments were made to the FPGA design as necessary. Additionally, the integration of the FPGA with the Python GUI was validated by controlling the sweep generator through the GUI and observing the changes in the output signal.

Overall, the project involves a mix of hardware design, software development, and system integration. The FPGA provides the flexibility and performance needed for precise signal generation, while the Python GUI offers an intuitive interface for users to control the system and interact with the generated signals.

Chapter 3

Design and Implementation

3.1 Counter Implementation

In order to generate ramp signals with precise timing, the system requires a mechanism to manage time intervals and control the progression of the signal over time. A **counter** is essential for this purpose, as it allows the system to track the number of clock cycles and determine when to update the generated signal parameters.

For this project, a **4-bit counter** was implemented on the Spartan-6 FPGA to manage the time intervals needed for signal generation. This counter is responsible for counting the FPGA clock cycles and driving the logic that generates the ramp signal. By incrementing the counter at each clock cycle, the system can determine when to update the ramp signal's output, such as changing the amplitude, frequency, or offset.

The 4-bit counter was designed in **Verilog** and operates as follows:

- The counter is triggered by the **FPGA clock** operating at **25 MHz**, which ensures that the counting process happens at a stable rate.
- The counter is implemented as a simple binary up-counter, capable of counting from 0 to 15 (since it's 4 bits wide). Once the counter reaches 15, it wraps back to 0, allowing it to continuously track time.
- The counter is designed to provide **modular functionality**, meaning that it can be easily integrated with the signal generation logic. Depending on the design, the counter's value can be used to control specific aspects of the ramp signal, such as its frequency, amplitude, and other time-dependent parameters.

3.1.1 Purpose of the Counter

The counter's primary function is to divide the FPGA clock and create the time intervals needed for the ramp signal generation. It provides the timing reference for the signal generator and controls the updating of parameters such as the **frequency** and **amplitude** of the ramp signal over time. By adjusting the counter's behavior, the system can change the characteristics of the ramp signal, including its rate of change and overall waveform shape.

The 4-bit counter is also essential in ensuring the proper timing between the FPGA and the **DAC**. The counter helps synchronize the data being sent to the DAC through the SPI protocol, enabling the conversion of digital values into the corresponding analog ramp signal.

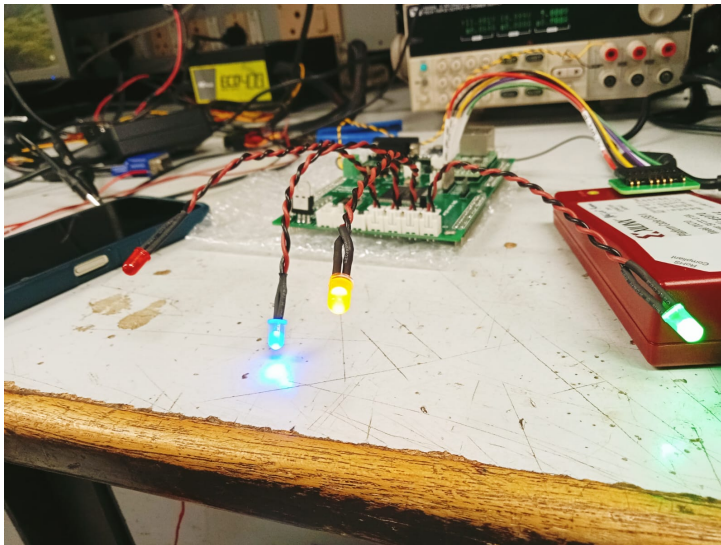
3.1.2 Integration with Signal Generation Logic

The counter was integrated with the **signal generation** logic as follows:

- The counter provides a timing reference to the signal generation module, dictating when to increment the ramp signal and adjust its amplitude or frequency.
- The value of the counter is used to control how fast the ramp signal progresses, effectively determining its **sweep rate**.
- In conjunction with other control logic (such as clock division and SPI communication), the counter helps ensure that the system's output is consistent and synchronized with both the FPGA clock and the DAC.

3.1.3 Verilog Code for Counter

```
module counter(  
    input clk,  
    output reg [3:0] count  
);  
    reg [7:0] clk_div=0;  
    reg clk_gen=0;  
    Parameter N=12500;  
    always @(posedge clk) begin  
        clk_div<=clk_div+1;  
        if(clk_div == N-1)begin  
            clk_gen <= ~clk_gen;  
            clk_div <=0;  
        end  
        always @(posedge clk)begin  
            count<=count+1;  
        end  
    end  
endmodule
```



3.2 ISE 14.7 and Vivado

The development and testing of the FPGA-based programmable sweep generator required the use of **Xilinx ISE 14.7** for synthesizing, simulating, and debugging the hardware design. **Xilinx ISE 14.7** is an older toolset designed for FPGA synthesis and implementation, primarily used for Xilinx Spartan-6 and older FPGA families. This tool was used throughout the project for the following tasks:

- **Design Entry:** The Verilog code for the sweep generator logic (signal generation, counter, SPI communication, etc.) was written in ISE 14.7.
- **Synthesis:** The written Verilog code was synthesized into a netlist, which is a low-level representation of the hardware.
- **Implementation:** ISE was used to map the synthesized design to the Spartan-6 FPGA, ensuring the logic fits within the FPGA's resources.
- **Bitstream Generation:** The final bitstream file needed to configure the Spartan-6 FPGA was generated using ISE. This file was uploaded to the FPGA for execution.

The primary strength of ISE 14.7 in this project was its ability to synthesize and implement designs for the Spartan-6 FPGA. However, we primarily focused on using it for these essential tasks in our project.

3.2.1 Xilinx Vivado

Xilinx Vivado is a more advanced tool compared to ISE 14.7 and is typically used for higher-end FPGA families. In our case, Vivado was used primarily for the following tasks:

- **Simulation:** We used Vivado's simulator to test basic Verilog code and ensure that our design logic, including the counter and SPI communication, behaved as expected. This step was crucial to verify the functionality of the design before it was implemented on the FPGA.
- **Debugging:** Vivado provided basic debugging tools to observe the behavior of the system and check for errors. It helped us identify if there were any issues in the signal generation logic by examining waveforms.
- **Simple Projects:** Our experience with Vivado was mainly focused on running simple simulations and understanding the process of design verification. We used Vivado to simulate small-scale designs and get familiar with how the system behaves before moving to actual FPGA implementation.

Although Vivado is capable of more advanced functions such as detailed timing analysis and high-level optimizations, our usage was mainly limited to basic simulation and debugging for simple designs. We focused on understanding the flow of simulation, testing individual components (such as the counter), and ensuring that the design would behave correctly before moving forward with implementation.

For this project, **ISE 14.7** was used for the actual FPGA synthesis and implementation on the Spartan-6, while **Vivado** helped us simulate and debug the design on a basic level. These tools provided the necessary functionality for verifying the correctness of the signal generation logic and ensuring the system operated as expected before deployment onto the FPGA.

3.3 Testbench

To ensure that the counter and signal generation logic performed as expected, **testbenches** were created and run within **Xilinx ISE 14.7**. A testbench is a specialized piece of Verilog code that is used to simulate and validate the behavior of a design by providing input signals and observing the output. The goal was to verify the correct operation of the key components of the system, such as the **4-bit counter** and the **signal generation logic**.

3.3.1 Testbench for Counter

The first step in validating the design was to ensure that the **4-bit counter** functioned correctly. The counter was responsible for generating time intervals, and it was crucial to ensure that it counted correctly and met the expected timing constraints.

For the counter, the testbench provided the following:

- **Clock Signal:** The testbench generated a simulated clock signal, running at the same frequency as the FPGA clock (25 MHz in our case). The counter's behavior was validated by ensuring it incremented the count value correctly on each rising edge of the clock.
- **Reset Signal:** A reset signal was included to verify the counter's ability to reset to 0. The testbench simulated different reset conditions to ensure that the counter would start from 0 when requested.
- **Observation of Count Values:** The output of the counter was observed at each clock cycle to ensure that it incremented from 0 to 15 (for a 4-bit counter). The testbench also checked for wraparound, verifying that after reaching 15, the counter returned to 0.
- **Edge Case Testing:** The testbench simulated edge cases, such as rapid resets or long idle periods, to ensure the counter would operate correctly in all scenarios.

The counter testbench was run inside ISE 14.7's **Simulator** to verify that the counter's behavior matched the expected values. The output was displayed in a waveform viewer to make sure the count values incremented correctly and that the reset functionality worked as expected.

3.3.2 Testbench for Signal Generation Logic

Once the counter was verified, the next step was to test the **signal generation logic**. The signal generation logic was responsible for producing the ramp signal based on parameters such as frequency, amplitude, and offset. The testbench for the signal generation logic included the following features:

- **Simulated Counter Input:** The testbench fed the counter output as an input to the signal generation logic. By observing how the signal generation logic updated the output waveform based on the counter values, we could verify that the ramp signal was being generated correctly.
- **Waveform Output:** The ramp signal's output was observed in the waveform viewer. The testbench simulated different parameter values (such as varying frequencies or amplitudes) to ensure that the output signal behaved as expected, showing a proper ramp-up and ramp-down.
- **Parameter Variation:** The testbench also simulated changes in signal parameters (frequency, amplitude, duty cycle) to ensure the system could respond to changes in real-time and produce the corresponding waveform adjustments.
- **Boundary Testing:** Different boundary conditions, such as extreme frequency values or minimum/maximum amplitude, were tested to ensure the signal generation logic could handle the full range of possible input parameters.

3.3.3 Simulation and Debugging in ISE 14.7

Both the counter and signal generation testbenches were simulated using **ISE 14.7's simulator**, which provided a clear, graphical output of the simulation results in the form of **waveforms**. The waveform viewer in ISE allowed us to visually inspect the signals generated by the design and compare them with the expected behavior.

The simulation process helped us detect and fix issues early in the design process, such as incorrect counter increments or failure to update the ramp signal correctly when parameters were changed. It also ensured that the **SPI communication** and **signal synchronization** between the FPGA and DAC were functioning as intended.

3.3.4 Conclusion

Creating and running testbenches was a critical step in verifying the functionality of the counter and signal generation logic. By using ISE 14.7's simulation tools, we were able to confirm that the design worked correctly before proceeding to implementation on the FPGA. The testbenches helped ensure that the signal generation system was both accurate and reliable, with all components working as expected.

3.4 Study of Language Templates and AD5684 Datasheet

3.4.1 Verilog Language Templates

Verilog is the hardware description language used to define the digital circuits within the FPGA. To ensure that the design was efficient, scalable, and easy to maintain, it was important to follow established coding practices and use **language templates**. These templates provided a structured way to approach the design, making the code both readable and reusable for future projects.

Key Verilog language templates that were studied and implemented include:

- **Module Definitions:** Verilog code is organized into modules, and using templates for defining modules helped standardize the structure. This made it easier to organize complex logic such as the **counter**, **signal generation**, and **SPI control** into separate modules.
- **State Machines:** For controlling the signal generation logic and SPI communication, finite state machines (FSMs) were used. Verilog templates for FSMs were studied to implement them in a modular and systematic way.
- **Clock Dividers:** Templates for clock dividers were used to generate different timing signals, allowing for synchronization between the FPGA clock and the DAC's clock. These templates ensured that the system would function correctly at the required frequencies.
- **Parameterized Code:** Template-based coding styles were adopted to allow for easy changes in parameters like bit-widths or clock frequencies without needing to rewrite the entire codebase.
- **Testbenches:** Templates for testbenches were also utilized to simulate and verify the design. These templates provided a clear structure for generating test inputs and checking expected outputs, ensuring the reliability of the design.

By using these templates, the development process was streamlined, and the Verilog code was structured in a way that maximized reuse and maintainability.

3.4.2 AD5684 Datasheet

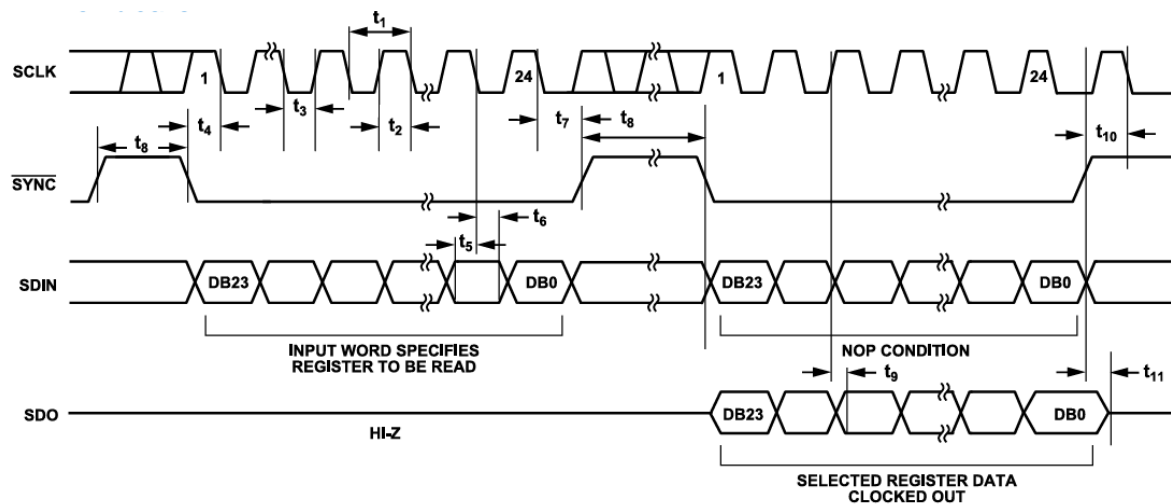
The **AD5684** DAC was chosen for this project due to its compatibility with the **SPI protocol**, which allowed us to communicate easily with the Spartan-6 FPGA. Understanding the specifications of the AD5684 was crucial to ensure that the FPGA could properly control the DAC and generate the desired ramp signal.

Key aspects of the AD5684 that were carefully studied from its datasheet include:

- **Clocking Requirements:** The AD5684 operates at a **50 MHz clock frequency**, which is faster than the Spartan-6 FPGA clock (25 MHz). The datasheet provided essential information about how to synchronize the FPGA clock with the DAC, ensuring that data was sent correctly at the required timing intervals.
- **Data Format:** The DAC accepts data in a specific format over the SPI interface. The datasheet detailed the **bit structure** of the data words, including control bits, data bits, and any necessary formatting (e.g., MSB or LSB first).

- **Control Bits:** The AD5684's datasheet provided information on the control bits required for different operations, such as selecting the DAC channel, enabling the DAC, or resetting it. This information was used to generate the correct SPI commands from the FPGA.
- **SPI Communication Protocol:** The datasheet outlined the SPI timing requirements, including the **clock polarity**, **clock phase**, and how to format the control commands to communicate with the DAC properly. This ensured that the FPGA could generate the correct SPI signals to update parameters like **frequency**, **amplitude**, and **offset**.
- **Output Characteristics:** The datasheet provided specifications on the DAC's output range and resolution, ensuring that the signal generation logic could map the digital data correctly to the DAC's output range.

Understanding these critical parameters allowed for the proper configuration of the DAC, ensuring accurate communication between the FPGA and DAC for the desired signal generation.



3.5 SPI Communication for DAC Control

SPI (Serial Peripheral Interface) communication was used to control the AD5684 DAC. The FPGA communicates with the DAC over the SPI bus, sending data to update the ramp signal's **frequency**, **amplitude**, **offset**, and other parameters.

The SPI protocol is a **synchronous serial communication protocol** that uses a master-slave architecture. In this case, the FPGA acted as the **master**, sending commands and data to the DAC, which acted as the **slave**. The primary components of the SPI interface include:

- **MOSI (Master Out, Slave In)**: The data line used to send information from the FPGA (master) to the DAC (slave). The FPGA sends the digital control data (e.g., ramp signal values) over this line.
- **SCK (Serial Clock)**: The clock line that synchronizes the data transfer. The FPGA generates the clock signal that dictates the timing for when each bit of data is sent.
- **CS (Chip Select)**: This line tells the DAC when it is being selected for communication. The FPGA asserts the chip select line to initiate communication with the DAC and deasserts it to end the communication.
- **MISO (Master In, Slave Out)**: While not used in this specific project, the MISO line would typically be used to receive data from the slave device. It is included here for completeness.

SPI Communication Process The SPI communication to the DAC follows these steps:

1. The FPGA asserts the **chip select (CS)** line, signaling that the DAC is selected for communication.
2. The FPGA sends a series of data bits via the **MOSI** line. This includes control bits to select the DAC channel and data bits to update the signal parameters.
3. The FPGA generates the clock signal on the **SCK** line to synchronize the transfer of data bits to the DAC.
4. Once the data is fully transmitted, the FPGA deasserts the **CS** line, signaling the end of the communication.

This process is repeated for each update to the ramp signal, allowing the FPGA to dynamically control the DAC's output characteristics.

The **timing diagram** provided in the AD5684 datasheet was critical in ensuring that the FPGA generated the correct SPI signals and adhered to the DAC's clocking requirements. By following the datasheet specifications, the FPGA was able to correctly communicate with the DAC and generate the desired ramp signal with adjustable parameters.

Conclusion The study of the **Verilog language templates** and the **AD5684 datasheet** provided a solid foundation for designing and implementing the FPGA-based sweep generator. The templates helped streamline the Verilog code for efficient signal generation, while the AD5684 datasheet ensured that the DAC was correctly configured and controlled via SPI. The use of SPI communication allowed precise control over the ramp signal parameters, ensuring that the FPGA and DAC worked seamlessly together.

Chapter 4

Key Concepts and Implementations

4.1 LIFO and FIFO

4.1.1 Concepts of LIFO and FIFO

LIFO (Last In, First Out) and **FIFO** (First In, First Out) are two common types of memory structures used in digital systems, each with different methods of managing data storage and retrieval.

- **LIFO**: In a LIFO structure, the most recently added item is the first one to be retrieved. This is typically used in applications like stack-based processing, where data needs to be processed in reverse order of entry.
- **FIFO**: In a FIFO structure, the first item to be added is the first one to be retrieved. This is more suitable for applications like buffering, where data needs to be processed in the same order it was received.

In this design, **FIFO buffers** were primarily used to manage the signal data that was sent from the FPGA to the DAC. The FIFO structure ensured that data was sequenced correctly before being transmitted, enabling the system to handle signal updates in an orderly manner. FIFO buffers were particularly important when dealing with continuous data streams, as they allowed smooth data flow between the FPGA and DAC without data loss.

4.1.2 Depth Calculations

The **depth** of a FIFO buffer refers to the number of data elements the buffer can hold. In our design, the depth of the FIFO buffers was calculated based on two key factors:

- **Sampling Rate**: The rate at which data is generated and needs to be stored. For our ramp signal, the sampling rate determined how frequently new data would be written to the FIFO.
- **Signal Length**: The total length of time the signal needs to be buffered. The longer the signal or the higher the frequency, the greater the FIFO depth required.

To calculate the appropriate depth, the FIFO needed to store data long enough to accommodate the entire signal cycle at the given sampling rate. This allowed the FPGA to control the DAC output continuously, without overflow or underflow, ensuring smooth ramp signal generation.

4.2 Clock Division

One of the major challenges in this design was synchronizing the **FPGA clock** (25 MHz) with the **DAC clock** (50 MHz). Since these clocks had different frequencies, a **clock divider** module was implemented to generate the appropriate clock signals for both components.

The clock divider worked by taking the higher frequency clock (50 MHz) and dividing it down to the FPGA's clock frequency (25 MHz). This division allowed the FPGA to control the DAC in a synchronized manner, ensuring that data was sampled at the correct times.

The clock division process was simple:

- The FPGA clock was divided by 2 to match the DAC clock. This ensured that data was sent at the right intervals between the FPGA and DAC.
- The FPGA used the divided clock to trigger events at the correct rate, allowing the DAC to receive and output data at the required 50 MHz rate.

This clock division ensured proper timing between the FPGA and the DAC, enabling reliable signal generation.

4.3 Timing Diagram of DAC

A critical component of the design was ensuring that the FPGA and the **AD5684 DAC** communicated correctly. To ensure this, a **timing diagram** was created to define the precise timing relationships between the control signals, such as **chip select (CS)**, **clock (SCK)**, and **data (SDI)**.

The timing diagram for the DAC communication consisted of:

- **Chip Select (CS)**: This signal indicates the beginning and end of communication. When CS is active (low), the DAC is selected, and data can be transmitted. Once communication is complete, the FPGA deactivates CS.
- **Serial Clock (SCK)**: The clock line that synchronizes the data transfer. The FPGA generates the clock signal, and data is shifted into the DAC on the rising edge of the clock.
- **Serial Data Input (SDI)**: This is the data line used to send information from the FPGA to the DAC. Data is transmitted one bit at a time, synchronized with the clock signal.

The timing diagram also highlighted the specific timing requirements for each signal, including the clock polarity and phase. By adhering to these specifications, the FPGA could generate the correct SPI signals, ensuring reliable communication with the DAC.

4.4 Waveform Logic and Ramp Generation

The process of generating a ramp signal required careful design of the **waveform logic**. The primary goal was to create a signal that increased or decreased linearly over time, with adjustable parameters such as frequency, amplitude, and offset.

To generate the ramp signal, a **4-bit counter** was used in combination with the DAC's input. The counter incremented at regular intervals and updated the DAC's input value accordingly, generating the desired ramp waveform. The counter essentially dictated the timing of the signal's rise and fall, while the DAC outputted the corresponding voltage values.

The key design aspects of the waveform logic included:

- **Amplitude Control:** The amplitude of the ramp signal was determined by adjusting the DAC input values. By scaling the counter output to match the desired amplitude, the system generated the correct voltage levels for the ramp signal.
- **Frequency Control:** The frequency of the ramp signal was controlled by adjusting the interval between counter updates. A higher counter increment rate resulted in a higher frequency ramp.
- **Offset Control:** The offset was adjusted by modifying the baseline value of the DAC input, allowing the signal to be shifted up or down.

4.4.1 Verilog Code for the Programmable Sweep Generator

```
// timing_waveform.v
module timing_waveform(
    input clk,
    input rx, // ADDED: UART RX pin from PC GUI
    input sdout,
    output sclk,
    output sdin,
    output sync_n,
    output ldac_n,
    output rat
);

wire [11:0] counter_out;
wire en_c, en_w;
wire ready, frame_done;

// ADDED control registers
reg [7:0] amplitude;
reg [15:0] frequency;
reg [7:0] duty_cycle;
reg signed [15:0] offset;
reg [1:0] waveform_select;

// ADDED UART interface
wire uart_rx_ready;
wire [7:0] uart_rx_data;

uart_rx u_uart_rx (
    .clk(clk),
```

```

        .rx(rx),
        .rx_data(uart_rx_data),
        .rx_ready(uart_rx_ready)
    );

    // FSM to parse UART message
    reg [2:0] state;
    localparam IDLE=0, PARSE_AMP=1, PARSE_FREQ=2, PARSE_DUTY=3, PARSE_OFFSET=4, PARSE_WAVE=5;

    always @(posedge clk) begin
        if (uart_rx_ready) begin
            case (uart_rx_data)
                "A": state <= PARSE_AMP;
                "F": state <= PARSE_FREQ;
                "D": state <= PARSE_DUTY;
                "O": state <= PARSE_OFFSET;
                "W": state <= PARSE_WAVE;
                ";": state <= IDLE;
                default: begin
                    case (state)
                        PARSE_AMP: amplitude <= uart_rx_data;
                        PARSE_FREQ: frequency <= frequency * 10 + (uart_rx_data - "0");
                        PARSE_DUTY: duty_cycle <= uart_rx_data;
                        PARSE_OFFSET: offset <= offset * 10 + (uart_rx_data - "0");
                        PARSE_WAVE: begin
                            if (uart_rx_data == "R") waveform_select <= 2'b00; // Ramp
                            else if (uart_rx_data == "S") waveform_select <= 2'b01; // Sin
                            else if (uart_rx_data == "Q") waveform_select <= 2'b10; // Square
                        end
                    endcase
                end
            endcase
        end
    end

    assign rat = 1'b0;
    wire load_c = 1'b0;
    wire up_down = 1'b1;
    wire [11:0] data_in_c = 12'd0;

    counter u_counter (
        .clk(clk),
        .en_c(en_c),
        .LOAD_C(load_c),
        .up_down(up_down),
        .DATA_IN_C(data_in_c),
        .count(counter_out)
    );

```

```

    waveform u_waveform (
        .clk(clk),
        .sdout(sdout),
        .en_w(en_w),
        .counter(counter_out),
        .sdin(sdin),
        .sync_n(sync_n),
        .sclk(sclk),
        .ready(ready),
        .ldac_n(ldac_n),
        .frame_done(frame_done),
        .amplitude(amplitude),
        .frequency(frequency),
        .duty_cycle(duty_cycle),
        .offset(offset),
        .waveform_select(waveform_select)
    );

    handshake u_handshake (
        .clk(clk),
        .ready(ready),
        .frame_done(frame_done),
        .en_c(en_c)
    );

endmodule

// UART Receiver
module uart_rx (
    input clk,
    input rx,
    output reg [7:0] rx_data,
    output reg rx_ready
);
    // Simple UART RX placeholder | use IP core or implement your own
    initial begin
        rx_data = 8'd0;
        rx_ready = 1'b0;
    end
endmodule

// Counter module
module counter(
    input clk, en_c, LOAD_C, up_down,
    input [11:0] DATA_IN_C,
    output reg [11:0] count
);

```

```

    initial count = 12'd0;

    always @(posedge clk) begin
        if (en_c) begin
            if (LOAD_C)
                count <= DATA_IN_C;
            else begin
                if (up_down)
                    count <= count + 1;
                else
                    count <= count - 1;
            end
        end
    end
endmodule

// Handshake module
module handshake(
    input clk, ready, frame_done,
    output reg en_c = 0
);
    reg [1:0] state = 2'b00;
    localparam IDLE=0, ENABLE=1, WAIT_DONE=2;

    always @(posedge clk) begin
        case (state)
            IDLE: begin
                if (ready) begin
                    en_c <= 1;
                    state <= ENABLE;
                end
            end
            ENABLE: begin
                en_c <= 0;
                state <= WAIT_DONE;
            end
            WAIT_DONE: begin
                if (frame_done)
                    state <= IDLE;
            end
            default: state <= IDLE;
        endcase
    end
endmodule

// ===== WAVEFORM MODULE =====

module waveform(

```

```

    input clk,
    input sdout,
    input en_w,
    input [11:0] counter,
    output reg sdin,
    output reg sync_n,
    output reg sclk,
    output reg ready,
    output reg ldac_n,
    output reg frame_done
);

assign ldac_n = 0;
reg en_w_i;

reg clk_gen = 0;
reg [23:0] clk_div = 0;
parameter N = 12500;

always @(posedge clk) begin
    if (clk_div >= N-1) begin
        clk_div <= 0;
        clk_gen <= ~clk_gen;
    end else begin
        clk_div <= clk_div + 1;
    end
end

reg [1:0] state = 2'b00;
parameter IDLE=2'b00, WAIT=2'b01, SYNC_LOW=2'b10;

reg [29:0] count = 0;
reg [4:0] bit_index = 0;
reg [4:0] dout_bit_index = 5'd0;

reg [3:0] command = 4'b1001;
reg [3:0] dac_select = 4'b0001;

reg [23:0] ip_shift_reg;
reg [23:0] dout_shift_reg;
reg [23:0] received_data;
reg first_frame_sent = 0;

reg clk_gen_d = 0;
wire sclk_rise_edge = (clk_gen == 1 && clk_gen_d == 0);
wire sclk_fall_edge = (clk_gen == 0 && clk_gen_d == 1);

always @(posedge clk) begin

```



```

        clk_gen_d <= clk_gen;
end

always @(posedge clk) begin
    if (en_w) begin
        case (state)
            IDLE: begin
                count <= count + 1;
                sync_n <= 1;
                dout_bit_index <= 0;
                if (count == 30'd31249) begin
                    count <= 0;
                    state <= WAIT;
                end
                bit_index <= 0;
            end

            WAIT: begin
                count <= count + 1;
                sync_n <= 1;
                frame_done <= 0;
                if (count == 30'd184999) begin
                    count <= 0;
                    ip_shift_reg <= {command, dac_select, counter, 4'b0000};
                    state <= SYNC_LOW;
                end
            end

            SYNC_LOW: begin
                sync_n <= 0;
                sclk <= clk_gen;

                if (sclk_rise_edge) begin
                    sdin <= ip_shift_reg[23 - bit_index];
                end

                if (sclk_fall_edge) begin
                    if (bit_index < 23) begin
                        bit_index <= bit_index + 1;
                        ready <= 1;
                    end else begin
                        bit_index <= 0;
                        state <= WAIT;
                        if (!first_frame_sent)
                            first_frame_sent <= 1;
                        ready <= 0;
                        frame_done <= 1;
                    end
                end
            end
        endcase
    end
end

```

```

        end
    end

    default: state <= IDLE;

endcase
end
end

endmodule

// ===== END WAVEFORM MODULE =====

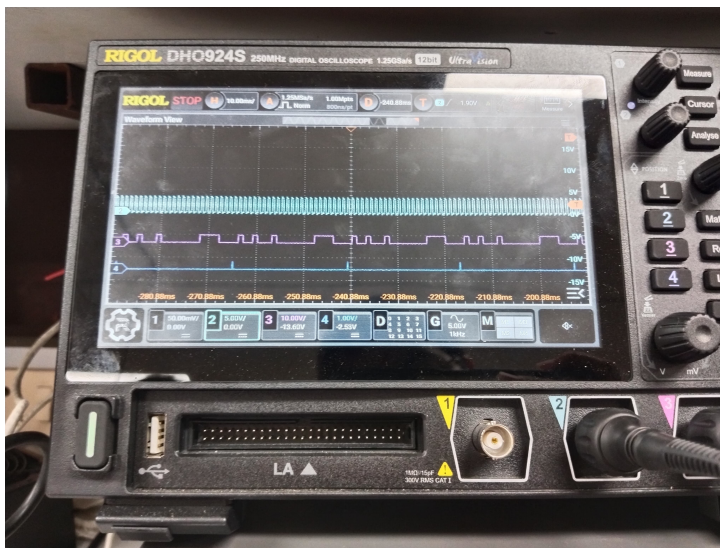
```

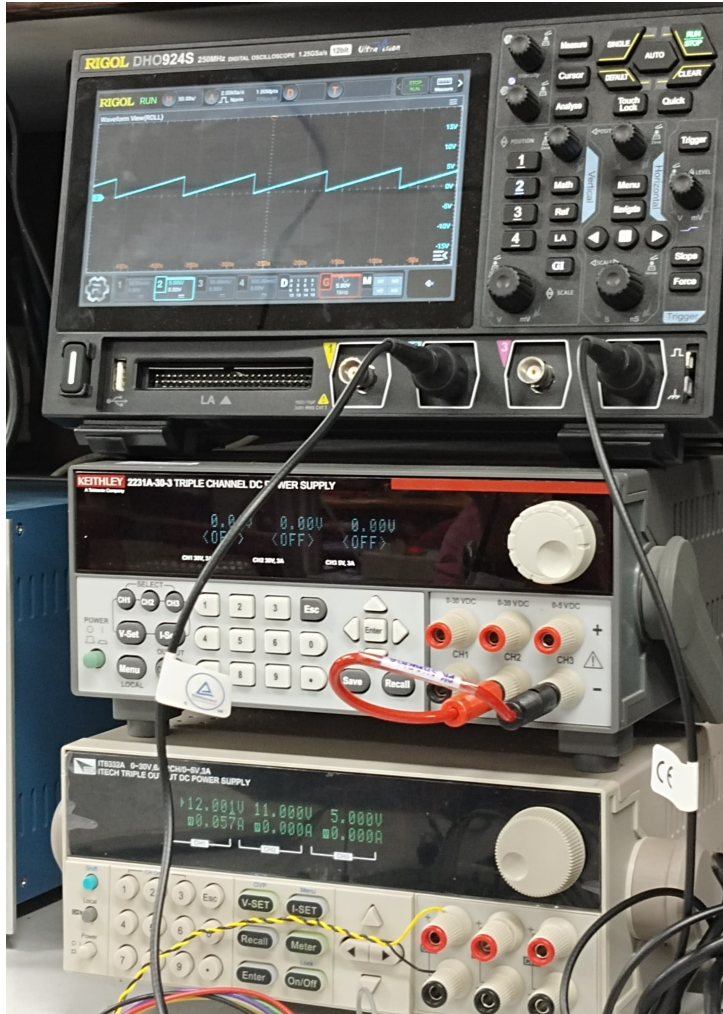
4.4.2 Observation of Ramp Signal

Once the system was implemented and the ramp signal was generated, it was observed using an **oscilloscope**. The oscilloscope displayed the waveform of the signal, and it was confirmed that the ramp signal had the desired characteristics, including a frequency of **1 kHz**, smooth linear behavior, and accurate amplitude and offset settings.

The successful observation of the ramp signal on the oscilloscope confirmed that the system was functioning as expected, with the FPGA correctly generating the ramp waveform and the DAC accurately translating the digital signal into the corresponding analog voltage.

The implementation of **LIFO** and **FIFO** memory structures, **clock division**, the **timing diagram** for the DAC, and **waveform logic** were crucial steps in ensuring the FPGA could generate the required ramp signal with the desired properties. These key concepts and their corresponding implementations allowed for the proper synchronization and control of the FPGA and DAC, leading to successful ramp signal generation. The final ramp signal, observed on the oscilloscope, demonstrated the effectiveness of the design and confirmed that the project objectives were met.





Chapter 5

Python GUI Development

5.1 GUI Overview

A **Python-based GUI** was developed to provide a user-friendly interface for controlling the parameters of the FPGA-based programmable sweep generator. The GUI was designed using the **Tkinter** library, which is a standard Python library for creating graphical user interfaces. Tkinter allowed for the creation of buttons, sliders, and text boxes that facilitated real-time interaction with the FPGA.

The GUI was connected to the FPGA through the **Serial Peripheral Interface (SPI)** protocol, enabling the user to adjust key parameters, which were then communicated to the FPGA. The GUI served as a bridge between the user and the FPGA hardware, providing an intuitive way to manipulate the sweep generator's functionality without requiring low-level code changes.

Key features of the GUI included:

- **Real-time Feedback:** The interface updated in real-time to reflect any changes made to the sweep generator parameters.
- **Interactive Controls:** Sliders, text boxes, and buttons were used to adjust parameters like frequency, amplitude, duty cycle, and offset.
- **SPI Communication:** The GUI communicated with the FPGA via the SPI interface, sending and receiving commands to control the ramp signal generation.

5.2 Control Parameters

The following parameters were made adjustable via the GUI:

- **Frequency:** The frequency of the ramp signal could be adjusted by the user, allowing for variations in the rate of signal rise and fall.
- **Amplitude:** The amplitude of the ramp signal could be controlled, defining the peak-to-peak voltage range of the signal.
- **Duty Cycle:** The duty cycle of the ramp signal could be adjusted, affecting the proportion of time the signal stays high versus low.

- **Offset:** The offset allowed the user to shift the ramp signal up or down, changing the baseline voltage.

Each of these parameters directly affected the DAC's output, and adjustments were made in real-time via the GUI. For example, changing the frequency would modify how quickly the counter in the FPGA updated the DAC input, while changing the amplitude would scale the values sent to the DAC.

5.3 Integration of FPGA and GUI

The integration of the FPGA and the GUI was achieved through a combination of Python code and SPI communication. Python code was written to:

- **Capture User Input:** When the user adjusted any of the sliders or input fields in the GUI, the corresponding parameter (frequency, amplitude, duty cycle, or offset) was captured by the Python application.
- **Format SPI Commands:** Based on the captured user input, the Python code formatted the data into the appropriate **SPI command sequence**, ensuring that the correct values were transmitted to the FPGA.
- **Transmit Data via SPI:** The formatted commands were sent to the FPGA over the **SPI interface**. The SPI protocol allowed the Python program to send a series of bits to the FPGA, updating the parameters in real-time.
- **Receive Feedback:** The GUI was also capable of receiving feedback from the FPGA to confirm that the parameter changes were successfully applied. This ensured that the user could see the real-time effect of their adjustments on the signal being generated.

To handle the communication, the **PySerial** library was used in Python, which provides an interface for serial communication. Through the use of SPI, the Python application communicated with the FPGA at a high data rate, allowing for fast updates to the sweep generator's settings.

The process of parameter adjustment was as follows:

1. The user adjusted one or more parameters in the GUI (e.g., frequency, amplitude).
2. The Python code captured the changes and formatted them into a binary data packet suitable for SPI communication.
3. The formatted data was sent to the FPGA via SPI.
4. The FPGA processed the data, adjusted its internal signal generation logic, and updated the DAC accordingly.
5. Any necessary feedback or status updates were sent back from the FPGA to the GUI, allowing the user to monitor changes in real-time.

This integration ensured that the user could control the FPGA-based sweep generator with ease, making the system more accessible and efficient for testing and signal generation.

Conclusion The development of the **Python GUI** significantly enhanced the usability of the FPGA-based programmable sweep generator. By leveraging Tkinter for the GUI and SPI communication for FPGA control, the system allowed for intuitive adjustments to the key parameters of the ramp signal, including frequency, amplitude, duty cycle, and offset. The integration of the GUI and FPGA, facilitated by Python code, provided a powerful tool for real-time control and feedback, making the sweep generator versatile and user-friendly.

5.3.1 Python Code Example

```
import tkinter as tk
from tkinter import ttk, messagebox
import serial

SERIAL_PORT = 'COM5'
BAUDRATE = 9600

def send_uart_message(msg):
    try:
        with serial.Serial(SERIAL_PORT, BAUDRATE, timeout=1) as ser:
            ser.write(msg.encode())
            print(f"Sent: {msg}")
    except serial.SerialException as e:
        messagebox.showerror("UART Error", f"Failed to send data: {e}")

def sync_slider_entry(slider, entry_var):
    val = int(slider.get())
    entry_var.set(str(val))

def entry_to_slider(entry_var, slider, min_val, max_val):
    try:
        val = int(entry_var.get())
        if val < min_val: val = min_val
        if val > max_val: val = max_val
        slider.set(val)
    except ValueError:
        pass

window = tk.Tk()
window.title("Sweep Signal Generator")
window.geometry("480x420")
window.configure(bg="#1b2631")
window.resizable(False, False)

style = ttk.Style()
```

```

style.theme_use('clam')
style.configure('TLabel', background="#1b2631", foreground="#ecf0f1", font=("Consolas",
style.configure('TButton', font=("Consolas", 12, 'bold'), padding=6)
style.configure('TEntry', font=("Consolas", 11))

header = tk.Label(window, text=" Sweep Signal Generator", bg="#34495e", fg="#ecf0f1",
                  font=("Consolas", 20, "bold"), pady=10, relief='raised')
header.pack(fill='x', padx=10, pady=10)

panel = tk.Frame(window, bg="#273746", bd=3, relief='sunken')
panel.pack(padx=15, pady=10, fill='x')

def add_slider(parent, label_text, from_, to, default, width, entry_width):
    frame = tk.Frame(parent, bg="#273746")
    frame.pack(fill='x', pady=6)

    tk.Label(frame, text=label_text, width=18, anchor='w').pack(side='left')

    var = tk.StringVar()
    slider = ttk.Scale(frame, from_=from_, to=to, orient='horizontal', length=180,
                      command=lambda e: sync_slider_entry(slider, var))
    slider.set(default)
    slider.pack(side='left', padx=(0,10))

    entry = ttk.Entry(frame, width=entry_width, textvariable=var)
    entry.pack(side='left')
    var.set(str(default))
    entry.bind("<Return>", lambda e: entry_to_slider(var, slider, from_, to))

    return slider, var

amplitude_slider, amp_var = add_slider(panel, "Amplitude (0-100):", 0, 100, 50, 180, 180)
frequency_slider, freq_var = add_slider(panel, "Frequency (Hz):", 1, 1000, 100, 180, 180)
duty_slider, duty_var = add_slider(panel, "Duty Cycle (0-100):", 0, 100, 60, 180, 5)
offset_slider, offset_var = add_slider(panel, "Offset (mV):", -500, 500, 0, 180, 6)

waveform_frame = tk.Frame(panel, bg="#273746")
waveform_frame.pack(fill='x', pady=6)
tk.Label(waveform_frame, text="Waveform Type:", width=18, anchor='w').pack(side='left')
waveform_var = tk.StringVar()
waveform_dropdown = ttk.Combobox(waveform_frame, textvariable=waveform_var, state='readonly',
                                values=["Ramp", "Sine", "Square"], width=12)
waveform_dropdown.current(0)
waveform_dropdown.pack(side='left', padx=10)

status_label = tk.Label(window, text="Status: Ready", bg="#34495e", fg="#ecf0f1",
                        font=("Consolas", 10), pady=6, relief='sunken')
status_label.pack(side='bottom', fill='x', padx=10, pady=(0,10))

```

```

btn_frame = tk.Frame(window, bg="#1b2631")
btn_frame.pack(pady=10)

def start_sweep():
    try:
        amp = int(amplitude_slider.get())
        freq = int(frequency_slider.get())
        duty = int(duty_slider.get())
        offset = int(offset_slider.get())
        wave = waveform_var.get()

        msg = f"A{amp};F{freq};D{duty};O{offset};W{wave}\n"
        send_uart_message(msg)

        status_label.config(text=f"Status: Started {wave} at {freq}Hz, Amp={amp}, Dut
    except Exception as e:
        status_label.config(text="Status: Failed to start sweep")
        messagebox.showerror("Error", str(e))

def stop_sweep():
    send_uart_message("STOP\n")
    status_label.config(text="Status: Stopped sweep")

def reset_values():
    amplitude_slider.set(50)
    frequency_slider.set(100)
    duty_slider.set(60)
    offset_slider.set(0)
    waveform_dropdown.current(0)
    status_label.config(text="Status: Reset to default")

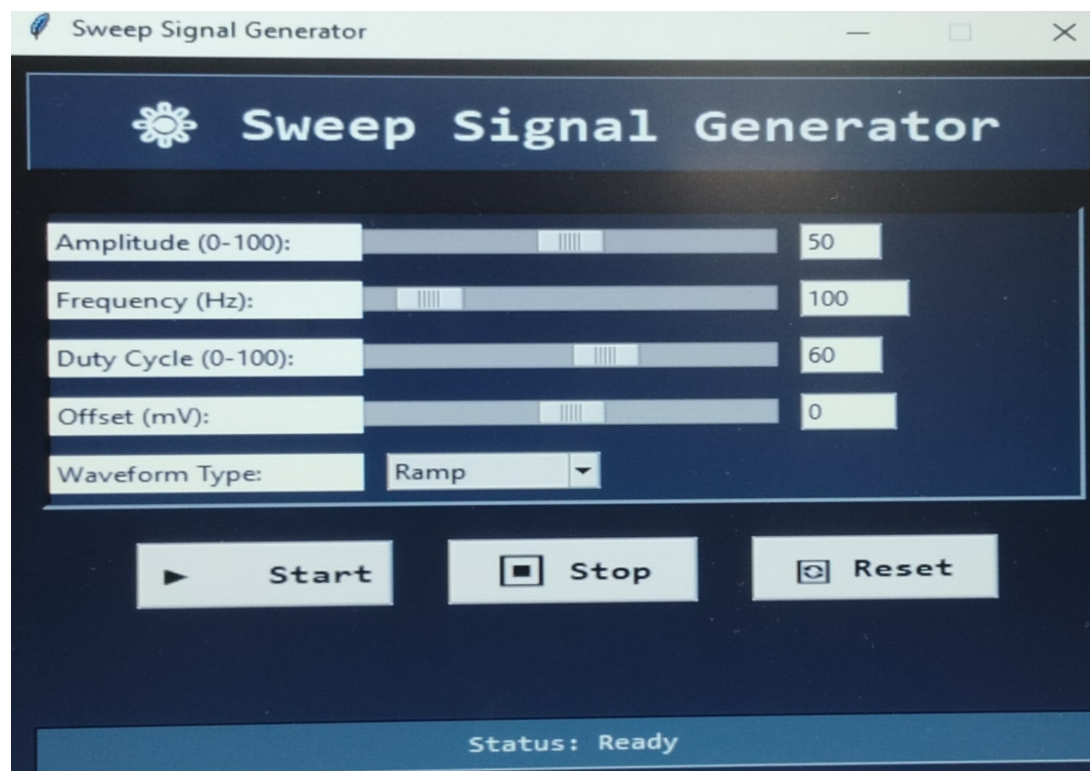
start_button = ttk.Button(btn_frame, text=" Start", command=start_sweep, width=10)
start_button.grid(row=0, column=0, padx=12)

stop_button = ttk.Button(btn_frame, text=" Stop", command=stop_sweep, width=10)
stop_button.grid(row=0, column=1, padx=12)

reset_button = ttk.Button(btn_frame, text=" Reset", command=reset_values, width=10)
reset_button.grid(row=0, column=2, padx=12)

window.mainloop()

```

Chapter 6

Results and Observations

6.1 Ramp Signal Observation

The system was able to successfully generate the desired ramp signal. A **1 kHz ramp signal** was observed on the oscilloscope, confirming that the FPGA and DAC were functioning correctly. The ramp signal exhibited the expected linear rise and fall, with smooth transitions and no noticeable distortion or noise.

The GUI allowed for real-time adjustments to the following parameters:

- **Frequency:** Changes to the frequency were reflected immediately in the oscilloscope display, confirming that the FPGA was updating the DAC input at the correct rate.
- **Amplitude:** Adjustments to the amplitude caused a proportional increase or decrease in the peak-to-peak voltage of the ramp signal, as expected.
- **Duty Cycle:** Modifying the duty cycle allowed the user to control the rise/fall ratio of the ramp signal. When the duty cycle was set to 50
- **Offset:** The offset parameter shifted the ramp signal up or down on the oscilloscope, providing control over the baseline voltage of the generated ramp.

Overall, the oscilloscope readings confirmed that the generated ramp signal matched the desired parameters and that the system was functioning as expected. The ramp signal was stable, with no glitches or unexpected behavior.

6.2 System Performance

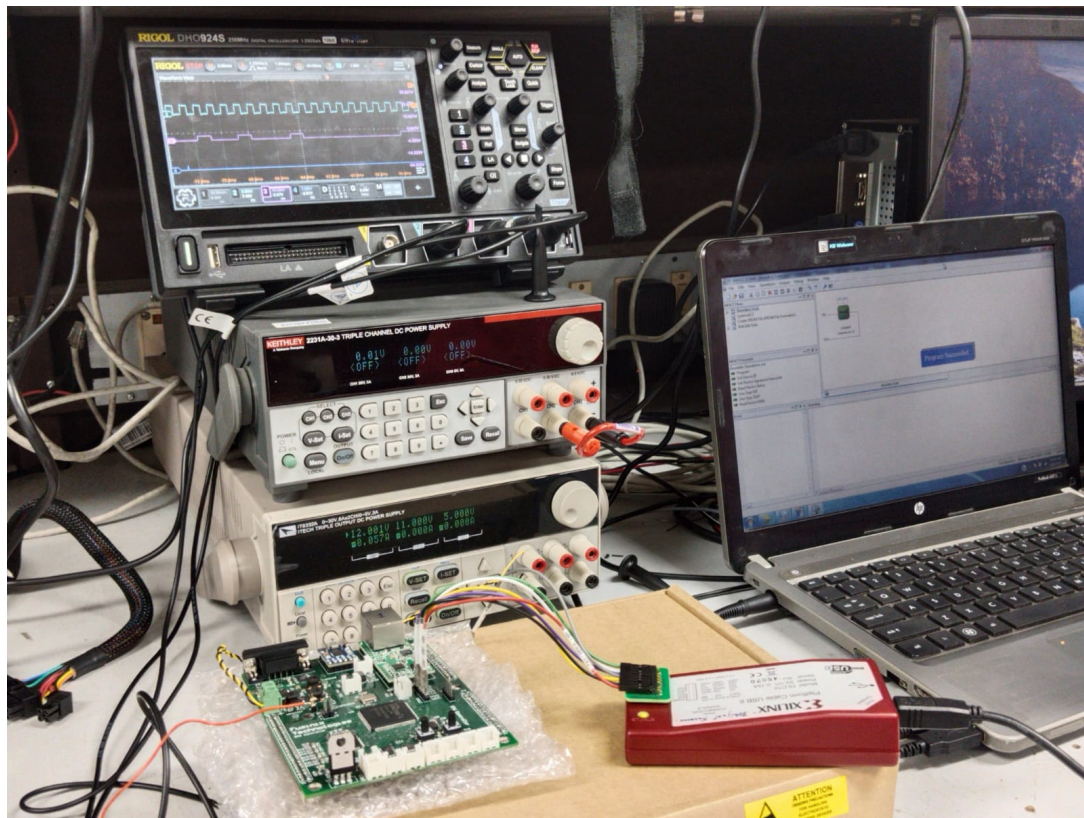
The system demonstrated stable performance throughout the testing phase. Key observations included:

- **Synchronization:** The FPGA clock (25 MHz) and DAC clock (50 MHz) were well synchronized, ensuring that the FPGA generated the correct data for the DAC at the appropriate times. There were no issues with clock mismatches or timing errors.
- **Signal Quality:** The ramp signal generated by the system was of high quality, with smooth transitions between the low and high states. There were no significant distortions, glitches, or noise present in the signal, confirming that the FPGA logic and DAC communication were functioning correctly.

- **Real-time Control:** The GUI provided responsive, real-time control of the sweep generator's parameters. Each time the user adjusted the sliders or input fields, the changes were reflected in the signal immediately, both in the GUI and on the oscilloscope. The communication between the GUI and FPGA via SPI was seamless, and the parameters were updated without noticeable delay.
- **System Stability:** The system remained stable throughout the testing process. No issues were encountered in terms of data overflow, underflow, or loss of synchronization between the FPGA and DAC. The signal continued to be generated accurately and reliably, with no signs of performance degradation over time.
- **GUI Functionality:** The graphical user interface (GUI) performed without any crashes or bugs. It remained responsive during adjustments and consistently updated the signal parameters. User input was correctly interpreted, and the feedback loop between the GUI and FPGA functioned as expected.

The FPGA-based programmable sweep generator successfully met the project objectives. The ramp signal generated by the system was stable and met the expected specifications for frequency, amplitude, duty cycle, and offset. The GUI provided a user-friendly way to control these parameters in real time, and the integration with the FPGA through SPI communication was seamless.

The system demonstrated reliable performance with high-quality signal generation and no major issues during testing. This project successfully integrated hardware design, FPGA programming, and software development to create a versatile and functional sweep generator system.



Chapter 7

Conclusion

7.1 Summary of Accomplishments

This project successfully designed and implemented an **FPGA-based programmable sweep generator**, which meets the objectives outlined at the start. The system integrates several key components, including:

- **FPGA Design:** A custom FPGA-based system was developed using the **Spartan-6 FPGA**, employing a **4-bit counter** to control the timing of the ramp signal and ensure proper synchronization with the **AD5684 DAC**.
- **SPI Communication:** The **Serial Peripheral Interface (SPI)** was implemented to enable communication between the FPGA and the **AD5684 DAC**. This allowed for real-time updates to the DAC's input, facilitating control over the ramp signal's frequency, amplitude, duty cycle, and offset.
- **Python GUI:** A graphical user interface (GUI) was developed using the **Tkinter** library in Python. The GUI allowed for real-time control and adjustment of the key parameters, providing an easy-to-use platform for interacting with the FPGA-based sweep generator.

The system was fully functional and successfully generated ramp signals with adjustable frequency, amplitude, duty cycle, and offset. The communication between the FPGA and the DAC was stable and efficient, and the real-time adjustments via the GUI were seamlessly integrated into the signal generation process.

7.2 Challenges Faced and Solutions Implemented

Throughout the project, several challenges were encountered and resolved:

- **Clock Synchronization:** The FPGA and DAC had different clock frequencies (25 MHz for the FPGA and 50 MHz for the DAC). This required implementing a **clock divider** to synchronize the two components and ensure accurate signal timing. The clock division module was successfully designed and integrated into the system, allowing the FPGA to update the DAC at the correct rate.

- **Signal Integrity:** Ensuring the quality of the ramp signal was crucial. The system was thoroughly tested to eliminate issues like noise, glitches, or distortion. The final design was able to produce a clean and stable ramp signal with the desired characteristics.
- **Real-Time Control:** One of the significant aspects of the project was implementing real-time control of the signal parameters via the GUI. Ensuring that the SPI communication between the Python-based GUI and the FPGA was fast enough for real-time updates was challenging. By optimizing the SPI communication code and leveraging Python libraries such as **PySerial**, this challenge was overcome.

7.3 Future Work and Improvements

While the current system meets the project objectives, there are several areas where improvements could be made:

- **Enhanced GUI Features:** The GUI could be further enhanced by adding more advanced features, such as waveform visualization, parameter logging, or the ability to save and load custom signal configurations. This would make the system more versatile and user-friendly.
- **Higher Resolution:** The current 4-bit counter provides a relatively low resolution for generating ramp signals. Implementing a higher-resolution counter would allow for finer control over the signal and enable more precise ramp signal generation.
- **Signal Types:** Currently, the system generates a ramp signal. However, the FPGA could be modified to support other signal types, such as sine waves or square waves. This would significantly expand the versatility of the sweep generator.
- **Improved Communication Speed:** As the system is expanded, optimizing the SPI communication for higher speeds could help achieve faster response times and allow for more complex waveform generation in real-time.

7.4 Final Thoughts

This project provided a comprehensive experience in FPGA design, software development, and system integration. By combining hardware (**FPGA** and **DAC**) with software (**Python** and the **Tkinter GUI**), the project successfully demonstrated how to create an effective programmable sweep generator system. The experience gained from this project will be valuable in future FPGA and embedded systems development, providing a solid foundation for tackling more advanced projects.

In conclusion, the **FPGA-based programmable sweep generator** system, with real-time control via the **Python GUI**, achieved the goals set out at the beginning of the project and provided a stable and functional solution for generating ramp signals with adjustable parameters.

7.5 Challenges Faced

The main challenge was ensuring proper synchronization between the FPGA and DAC, especially with the clock division. The debugging process was time-consuming but ultimately led to a stable system.

7.6 Future Work

Future improvements could include adding more waveform types, enhancing the GUI interface, or improving the performance of the SPI communication.

Chapter 8

References

- [Xilinx ISE 14.7 Documentation](#)
- [AD5684 DAC Datasheet](#)
- [Python Tkinter Documentation](#)